

Web Application Attack Tools

The interception paradigm

Almost every web test starts with a **proxy**: your browser talks to a local proxy that talks to the target. You see and change every request.

```
[ Browser ] -> [ Burp/ZAP Proxy :8080 ] -> [ Target ]
```

Burp Suite Community (free)

Proxy + Intercept: pause requests, modify, forward

Repeater: handcraft requests over and over

Intruder: fuzz payloads across parameters/hosts (Community limits speed)

Decoder/Comparer: encode/decode and diff responses

OWASP ZAP (free, open source)

Same proxy model + **active scan** + **fuzzing** + automated spider

Great as the DAST tool in CI pipelines too

Scriptable (Python/Zest) for repeated flows

The web attack tools corner

```
# command injection detection
commix --url 'http://10.0.0.20/cmd.php?cmd=whoami'

# parameter discovery (hidden params = attack surface)
arjun -u http://10.0.0.20/search.php

# fuzz directories/files
ffuf -u http://10.0.0.20/FUZZ -w /usr/share/wordlists/dirb/common.txt
```

```
# XSS detection
xssstrike -u 'http://10.0.0.20/?q=' -l -d # blind-ish probes

# SQLi deep
sqlmap -u 'http://10.0.0.20/?id=1' --batch --technique=BEUQT
```

Web workflow using Burp

Configure browser → Burp proxy

Browse the app, watch requests appear

Repeater: tamper with parameters, headers, body

Intruder/Wfuzz: fuzz known-bad payloads (sqlmap, XSS, LFI lists)

Record evidence for the report

The professional web stack

Never just "run sqlmap". Understand each request: where auth lives, what's stateless, how errors reveal structure. The tools accelerate a human who reads the traffic.

[← Wireless Attack Tools](#)

[Network & Traffic Analysis Too...](#)